

# ReCOR RAG: Hierarchical Indexing Based Grounded Conversational Question Answering

Team 65

**Abstract**—Analyzing large volumes of customer service call transcripts has become essential for businesses, as these conversations often lead to critical outcomes such as customer escalations or shifts to competitors. However, manually reviewing thousands of transcripts is infeasible and existing tools provide limited support for obtaining reliable evidence-based insights. To address this gap, we propose ReCOR RAG, a novel Retrieval-Augmented Generation (RAG) based framework designed to automatically identify the most relevant conversational segments and generate causal, evidence-backed explanations with the capability to continue conversations. We also contribute a set of analyst-oriented queries constituting both independent and follow up queries that enable more structured and in-depth exploration of conversational data. In order to rigorously assess our system and the curated queries, we also introduce new evaluation metrics specifically tailored for this task.

## I. INTRODUCTION

Retrieval-Augmented Generation (RAG) systems [1] are being integrated into nearly every sector, but their latent potential to efficiently understand and solve complex customer inquiries within vast repositories of conversational data is widely unexplored. Among the various forms of business data, customer service conversation transcripts hold particular significance, as they capture rich, unfiltered insights into customer experiences, concerns, and decision-making processes. Analyzing conversational patterns enables businesses to uncover operational gaps, understand customer sentiment, identify pitfalls leading to escalation or refund, detect early signs of churn, and respond quickly to critical interaction outcomes.

Despite their importance, manually reviewing large repository of transcripts is extremely challenging. It is challenging for an analyst to study tens of thousands of noisy conversations. Branching dialog structure complicates detection of causal spans, and retrieval over such large corpora is often difficult to scale. Clearly, there is critical need for scalable, consistent, and context-aware systems capable of supporting analysts in identifying conversational signals and explaining adverse outcomes. Additionally, most current systems operate at the level of a single conversation and fail to perform robust aggregate inference across multiple related transcripts.

We address the above highlighted gaps by proposing a robust framework capable of providing reliable, evidence-grounded causal explanations across a corpus of multi-intent conversations. Our system incorporates **RAPTOR-style [2] hierarchical summarization** equipped with a novel **reranking algorithm**. We then deliberately separate the reasoning and execution phases of a query task via the **ReWOO [3] framework** to further enhance the accuracy and efficiency of

the final generated response. Our thorough experimentation and investigation revealed the accuracy benefits of using RAPTOR, however, at the cost of a large average response time per query. To address this, we integrated **vLLM [4]**, an inference server that speeds up the output generation time using PagedAttention, an attention algorithm inspired by the classical virtual memory and paging techniques present in existing operating systems. Furthermore, **dynamic batching** is done to parallelize the summary generation of clusters at each level of the RAPTOR tree, making better use of GPU memory and optimizing the build time.

Apart from making a conversational reasoning system to analyze the data and provide actionable insights, this work also provides structured analytical queries that can capture different types of reasoning ( eg. Descriptive, Root Cause Analysis, Understanding Behavioral Patterns) for business-critical events such as escalations, refunds, and churn along with context-aware follow-up queries. Large Language Models (LLMs) often produce queries that lack reasoning clarity, domain grounding, or logical diversity, leading to redundancy, weak category alignment, and hallucinated causal links. The challenge, therefore, lies in designing a structured query-generation framework that can reliably produce realistic, grounded, reasoning-aligned questions to rigorously test and evaluate the systems performance as well as effectively analyzing the rich conversational data through the breadth of the generated queries.

To address these gaps, we propose a **Taxonomy-Guided Query Generation and Evaluation Framework** that ensures comprehensive reasoning coverage while maintaining realism and diversity. The framework comprises three components: (1) a **MECE-based taxonomy**, inspired by classical analytical paradigms (Descriptive, Diagnostic, Comparative, and Contextual) to structure queries by reasoning intent and complexity; (2) a **three-stage generation pipeline** implemented via LLMs covering intent/event generation, analytical query generation (Task 1), and contextual follow-up generation (Task 2); and (3) a **multi-level evaluation system** combining **non-LLM metrics** (grammar, readability, lexical and semantic similarity) with **LLM-as-a-Judge** scoring calibrated using a human gold dataset. The generated queries were filtered, difficulty mismatches reclassified, yielding a high-quality, reasoning-aligned, and domain-grounded analytical query dataset.

## II. CONTRIBUTION

Through our report we make the following contributions:

- We identify and address the key challenges involved in efficiently analyzing a large corpus of conversational transcripts, highlighting the limitations of existing tools.
- We propose **ReCOR RAG**, a framework that extends conventional RAPTOR by building the hierarchy dynamically at query time, which are reranked by the ColBERT-reranker, generating a set of query-specific evidence documents, and enabling robust response generation via the ReWOO framework.
- We propose the solution of **Task 1** as described by the problem statement through the above mentioned novel framework and extend this to support the follow-up interactions along with rigorously evaluating this capability in **Task 2**.
- We introduce novel mechanisms for formulating analyst-oriented queries **with and without the complete knowledge of dataset**, allowing analysts to explore the dataset from multiple analytical perspectives.
- We conduct rigorous evaluation to demonstrate the effectiveness of our approach, including the introduction of unique metrics tailored to large-scale conversational analysis. and also describe in appendix where the conventional metrics fails and why.

### III. RELATED WORK

#### A. Why Do Traditional RAG Systems Fail on Conversational Question Answering?

Traditional RAG systems rely on contiguous chunking, where documents are split into fixed sized segments before being encoded and retrieved. This approach presents clear limitations as noted by [5], contiguous segmentation may fail to capture the full semantic depth of a document because meaningful ideas often span across chunk boundaries. This leads to fragmented context, retrieval drift, and reduced answer relevance.

To address the contextual fragmentation inherent in naive chunking, hierarchical and semantically informed chunking methods have been proposed. Among them, RAPTOR (Retrieval Augmented Generation through Hierarchical Clustering) has emerged as a powerful approach. RAPTOR constructs a multi-level hierarchical tree of summaries, enabling retrieval at varying levels of abstraction. Instead of relying solely on raw text chunks, RAPTOR retrieves semantically enriched summaries that preserves long contextual dependencies often present in call transcripts. This mechanism also significantly improves performance in tasks requiring global document understanding, such as question answering, summarization, and evidence documents used to generate the answer.

#### B. Some other retrieval approaches

Beyond chunking approaches, other lines of work focus on query expansion, context filtering, and reranking techniques (e.g., ColBERT). These models aim to refine retrieval outputs to reduce noise and improve faithfulness. Recent agentic RAG frameworks also incorporate multi step reasoning and dynamic retrieval to better generate human-friendly responses.

Hierarchy-based retrieval techniques besides RAPTOR have also been explored. GraphRAG [6] constructs relational knowledge graphs over documents to capture entity-level information and enable path-based retrieval. While effective for structured source datasets, GraphRAG struggles with noisy, two way conversations where relationships are not explicitly stated, not to mention its resource intensiveness. Other techniques like HyDE [7] and IRCOT [8] use hypothetical document expansion or Chain-of-Thought [9] reasoning to enrich retrieval queries, but these methods also tend to drift semantically on informal conversational corpora.

Improving retrieval precision has been another focus. Late interaction models such as ColBERT perform fine grained token level MaxSim matching and consistently outperform dense bi-encoders on tasks requiring nuanced relevance. Hybrid and multi retriever systems, such as MRAG [10], attempt to combine lexical and semantic signals to improve robustness. Despite these innovations, prior work has not addressed the combined challenge of:

- (i) Long context conversational structure
- (ii) Fine grained evidence grounding
- (iii) Cross conversation causal pattern and span extraction

Reasoning frameworks such as ReWOO separate planning from execution and enable transparent, tool driven reasoning to generate answers grounded in retrieved evidence. Yet, to the best of our knowledge, no existing system integrates hierarchical retrieval (as in RAPTOR), token-level reranking (as in ColBERT), and structured reasoning (as in ReWOO) into a unified pipeline tailored for large-scale conversational analysis.

### IV. CUSTOM CURATED QUERY DATASET

A wide and diverse dataset is essential to enable analysts to cover as many topics as possible. In order to curate such a dataset, we adopted a taxonomy-guided structured generation approach, similar to the TaP framework [11], which has shown that taxonomy-grounded prompting yields diverse, balanced, and scalable synthetic data generation via LLMs. To achieve this, a well-structured taxonomy is crucial. We adopted the **Mutually Exclusive and Collectively Exhaustive** (MECE) principle originally popularized by McKinsey & Company for problem structuring. This framework ensures that each query belongs to exactly one reasoning category (*mutual exclusion*), while all possible analytical reasoning types are represented (*collective exhaustion*). The taxonomy was aligned with four analytics types, i.e. Descriptive, Diagnostic, Predictive, and Prescriptive, but adapted for conversational reasoning as Descriptive, Diagnostic, Comparative, and Contextual. The subtypes were derived through a bottom-up process that ensured MECE consistency, uniqueness, coverage, and domain alignment. Queries were further classified by difficulty; Easy, Medium, and Hard based on reasoning depth, ranging from direct retrieval to multi-hop contextual and counterfactual analysis.

For query dataset curation, we followed two different methodologies; the first independent of the call transcript

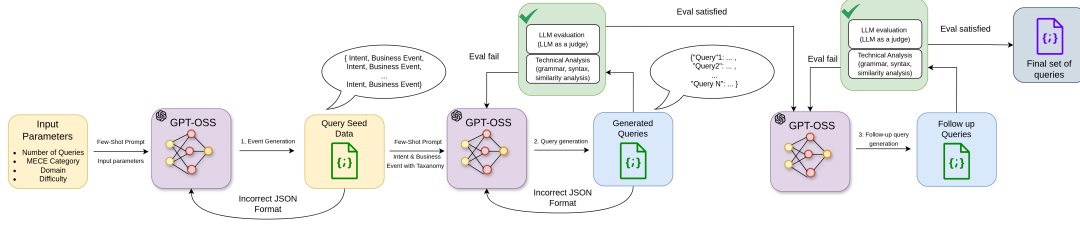


Fig. 1. Method 1: Automated multi-stage query generation and validation pipeline.

dataset and the second dependent on call transcript dataset, hence maintaining generalization of queries as well as groundedness.

**Method 1: Transcript-Agnostic Analytical Query Generation Pipeline** We used a three-stage approach:

- 1) **Stage-1: Generation of customer intents and key business events** A parameterized function accepts the number of queries  $Q$ , domain  $D$ , difficulty, and optionally MECE category and subcategory to control reasoning scope and precision as its inputs. When the category is unspecified, the generator automatically balances query distribution across the MECE types. Given a domain  $D$  and the desired number of queries  $Q$ . In the first stage of the pipeline the LLM is invoked with a structured few-shot prompting template that guides it to create **domain-specific intents** (using examples influenced from the given call transcript dataset to maintain groundedness), write **short intent descriptions**, and list 13 **measurable business events**. Outputs were enforced in strict JSON format using re-prompting for accuracy and automated evaluation (see Appendix for schema & the prompt used).
- 2) **Stage-2: Analytical Query Generation (For Task 1)** Out of the four MECE categories, Stage 2 is focused on generating analytical queries aligned with the Diagnostic, Descriptive, and Comparative categories of the MECE taxonomy. Using the domain, intent, and business event pairs from Stage 1, the model produces reasoning-driven questions to resemble real analyst inquiries. Only one MECE category and its subtypes were used while creating a query to reduce cognitive load and maintain structural clarity. The Stage 2 prompt template included the business event, target reasoning category, and difficulty level, and the invoked LLM generated one JSON object per event in the schema domain, business-event, mece-category, subcategory, query, difficulty. (see Appendix for details).
- 3) **Stage-3: Contextual Follow-Up Query Generation (For Task 2)** The final stage generated contextual follow-up queries that extended the analytical reasoning from Task 1, implementing the Contextual branch of the MECE taxonomy. The prompt template for this stage used domain, business event, original query, MECE

category, and a specified follow-up subtype (e.g., Evidence Retrieval, Clarification, Refinement, Counterfactual) from Stages 1 and 2, in order to produce one contextual follow-up per query.

**Method 2: Self-Instruct and Auto Evol-Instruct Query Evolution**

Method 2 constructs a difficulty-aware pool of analytical queries using a pipeline that combines Self-Instruct-style bootstrapping [12] with Auto Evol-Instruct-style controlled evolution [13], followed by lightweight evaluation. Unlike Method 1, which is explicitly taxonomy-driven, Method 2 focuses on domain-agnostic, transcript-answerable analytical queries and on systematically increasing their reasoning complexity.

**Stage 0: Seed generation:** We first generate 6 seed queries per each domain belonging to a set of predefined queries for the domains - Hotel, Flight, Retail, Banking, Telecom, Insurance using structured prompts that enforce: (i) answerability from call transcripts alone, (ii) length between 10-25 words, (iii) focus on behavioral, causal, or process-level analysis, and (iv) no dependence on SQL, transcript IDs, or external data sources. Seeds are generated using *llama-3.1-8b-instant*. The generated seed queries are labelled as  $S$ .

**Stage 1: Self-Instruct bootstrapping:** Starting from the seed set  $S$ , we iteratively expand a pool of queries  $Q$  using a Self-Instruct loop. At each iteration, we: (1) sample a small batch of examples from  $S \cup Q$ , (2) prompt the LLM to generate new analytical queries in a similar style, (3) filter candidates by length (7-35 words) and lexical similarity (Jaccard similarity), and (4) add only the filtered queries to  $Q$  until the target size is reached.

To evaluate the resulting pool, we compute four non-LLM metrics:

**Lexical diversity** is measured via average pairwise Jaccard distance:

$$D_{\text{lex}} = \frac{1}{|Q|^2} \sum_{i,j} \left( 1 - \frac{|w_i \cap w_j|}{|w_i \cup w_j|} \right),$$

where  $w_i$  is the set of (lowercased) tokens in query  $i$ .

**Semantic diversity** uses sentence embeddings (all-MiniLM-L6-v2) and cosine similarity:

$$D_{\text{sem}} = 1 - \frac{1}{|Q|^2} \sum_{i,j} \cos(\mathbf{e}_i, \mathbf{e}_j),$$

where  $\mathbf{e}_i$  is the embedding of query  $i$ .

*N-gram uniqueness* is defined as the ratio of unique trigrams to total trigrams across  $Q$ .

*Type-Token Ratio (TTR)* as :

$$\text{TTR} = \frac{|\text{unique words}|}{|\text{total words}|}.$$

Across six domains with 6 seeds we obtain the diversity profile summarized in Table 1.

| Metric                              | Value          |
|-------------------------------------|----------------|
| Mean length (words)                 | $12.9 \pm 1.8$ |
| Lexical diversity $D_{\text{lex}}$  | 0.941          |
| Semantic diversity $D_{\text{sem}}$ | 0.622          |
| N-gram uniqueness                   | 93.6%          |
| Type-Token Ratio                    | 0.508          |
| Domain coverage                     | 6 domains      |

**Table 1.** Stage 1 Self-Instruct diversity metrics for Method 2.

**Stage 2: Auto Evol-Instruct with dual voting.** Each base query from Stage 1 is then evolved to increase reasoning depth while preserving its analytical intent and domain context. We assign target difficulty levels - Easy (30%, 0 evolution rounds), Medium (50%, 12 rounds), and Hard (20%, up to 3 rounds) and apply an Auto Evol-Instructstyle template: The model (i) lists candidate evolution operators (e.g., contextual framing, scope refinement, analytical deepening), (ii) selects a plan, (iii) rewrites the query under strict length constraints and (iv) refines the final version.

To avoid trivial or low-quality rewrites, we use a dual-voting mechanism:

- A **function-based vote** applies deterministic checks: minimum length increase, non-decreasing syntactic depth (parse-tree depth), bounded semantic similarity with the original (BERTScore) [14], and non-decreasing Flesch-Kincaid readability.
- An **LLM-based vote** evaluates whether the evolved query preserves the original intent, increases analytical depth (e.g., adding causal or comparative structure), remains clear and RAG-friendly (answerable from transcripts alone), and does not hallucinate transcript IDs, dates, or domain-specific details.

We adopt a strict AND rejection rule: if either vote returns FAIL, the evolution is discarded or reattempted. In practice, this yields an evolution success rate of about 95% with high agreement between the two voters and produces a difficulty-stratified set of evolved queries where semantic divergence from the original increases with difficulty, thus introducing diversity in the generated query set, while remaining grounded and transcript-answerable.

**Lightweight evaluation.** Finally, we summarize evolution quality using (i) BERTScore and ROUGE-L [15] between original and evolved queries, and (ii) a compact G-Eval-style [16] LLM-as-a-judge rubric (complexity, RAG-friendliness, clarity, feasibility), as well as safety checks (bias, toxicity,

hallucination, faithfulness) implemented via DeepEval [17]. Aggregated scores indicate high preservation for Easy queries, controlled semantic drift for Medium/Hard queries, high average RAG-friendliness, and considerable pass rates across safety dimensions. Method 2 therefore acts as a compact query evolution engine that upgrades the base pool into a difficulty-balanced, safety-screened analytical benchmark.

**Method 3: FPS-Guided Curation and Multi-Scope Grounded Query-Answer Pairs** While Methods 1 and 2 focus on taxonomy-aligned, transcript-agnostic templates and their evolution, Method 3 is a transcript-first pipeline that operates directly on a large call transcript corpus. It combines **farthest-point sampling** (FPS) [18], summary-based clustering, and LLM-based queryanswer generation to produce fully grounded Task-1 and Task-2 pairs. The overall pipeline is illustrated in Fig. 2.

**Stage 1: FPS-based dataset curation:** Given a corpus  $\mathcal{D} = \{d_1, \dots, d_N\}$  of call transcripts, each transcript  $d_i$  carries structured metadata ( $\text{domain}_i, \text{intent}_i, \text{reason}_i$ ) and a raw conversation  $C_i$ . We first construct a metadata-rich string:

$$s_i = \text{concat}(\text{domain}_i, \text{"call regarding"}, \text{intent}_i, \text{":"}, \text{reason}_i)$$

We embed  $s_i$  using Sentence-BERT [19] (*all-MiniLM-L6-v2*) to obtain  $\mathbf{e}_i \in \mathbb{R}^{384}$ . For each domain, we (i) determine an approximate number of clusters  $K^*$  by scanning  $K$  and maximizing silhouette score [20], (ii) run K-means with  $K^*$  on the domain-specific embeddings, and (iii) apply FPS within each cluster to select a small set of diverse representatives (centroid-near prototypes and farthest edge cases). This yields a curated subset  $\mathcal{D}_{\text{FPS}}^*$  that reduces the corpus by roughly 85-90% while preserving domain coverage and rare but important behaviors.

**Stage 2: Summary-based semantic clustering:** To capture semantic nuances beyond metadata, we generate abstractive summaries for each transcript and embed them using the same encoder to obtain  $\mathbf{e}_i^{\text{summ}}$ . Grouping transcripts by (domain, intent), we repeatedly sample random anchors and select farthest neighbors in embedding space, forming intent-specific clusters enriched with semantically atypical conversations that centroid-based selection alone would miss. This produces a second curated subset  $\mathcal{D}_{\text{Intent}}^*$ , complementary to  $\mathcal{D}_{\text{FPS}}^*$ .

**Stage 3: Multi-scope queryanswer generation:** This method defines a query taxonomy over eight scopes: single-transcript (ST), cluster-aggregate (CA), cross-cluster (CC), cross-domain (CD), temporal (TM), causal-chain (CL), counterfactual (CF), and unanswerable (UN). Each query is tagged with a difficulty level (easy/medium/hard), a reasoning type (causal, temporal, comparative, conditional, behavioral), and a business analytics subtype (e.g., churn drivers, escalation causes, process breakdowns). ST queries primarily use  $\mathcal{D}_{\text{FPS}}^*$  to maximize coverage of diverse metadata configurations, whereas CA/CC/CD/TM/CL/CF queries leverage  $\mathcal{D}_{\text{Intent}}^*$  to reason over coherent yet diverse groups of conversations.

For each query  $q_j$ , an LLM is prompted with appropriate context (full transcripts, subsets of turns or cluster summaries)

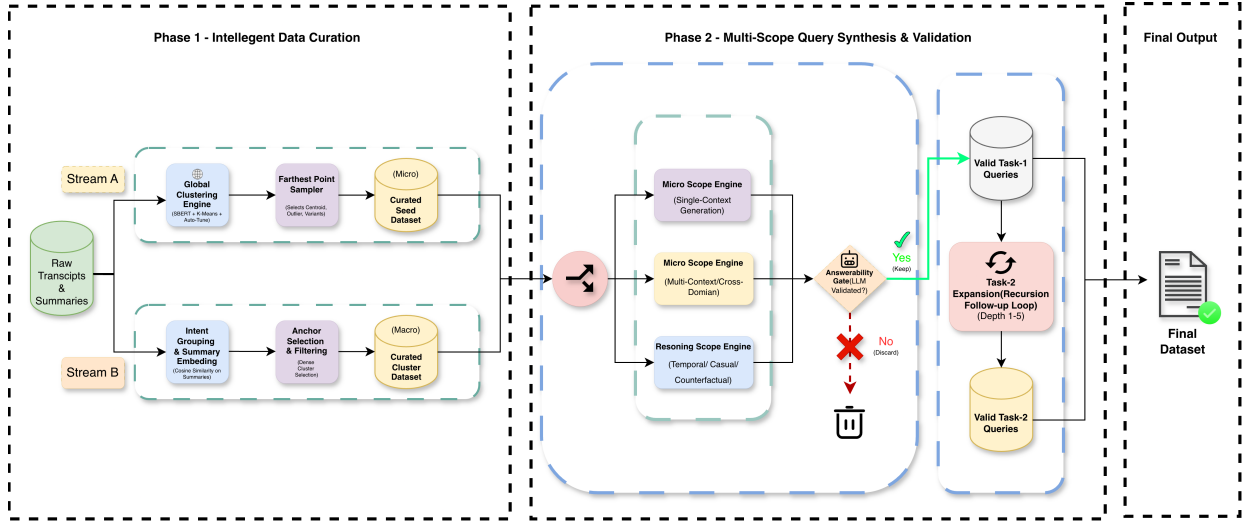


Fig. 2. Method 3: Stage 1 performs FPS-based curation on metadata embeddings to obtain a diverse subset of transcripts. Stage 2 performs summary-based semantic clustering and anchor-driven outlier discovery. Stage 3 uses both curated subsets as inputs to a multi-scope query-answer generation framework, producing single-transcript and aggregate queries (Task 1) and multi-turn follow-ups (Task 2) grounded in real call center data.

to generate a ground-truth style answer  $a_j$ . We enforce an explicit `INSUFFICIENT_DATA` convention and discard any query-answer pair where the answer cannot be adequately supported by the provided evidence. Generated pairs are then evaluated using faithfulness and answer-correctness checks (DeepEval), semantic similarity (BERTScore), and lexical overlap (ROUGE-L). For a subset of Task-1 queries (around 60%), we further generate Task-2 follow-up questions conditioned on  $(q_j, a_j)$  and prior turns, ensuring that follow-ups introduce new analytical angles (clarification, refinement, contrast) while staying grounded in the same evidence.

In such a way, we explored the process of query generation both with and without reference call transcripts to create a comprehensive dataset encompassing the breadth of reasoning types found in contemporary works.

## V. RAG SYSTEM

The core hypothesis underlying ReCOR RAG is that customer conversations exhibit significant latent structure. Many transcripts share similar *reasons for contact*, *failure modes*, and *outcomes*, motivating us to adopt hierarchical approaches that can effectively capture and leverage these recurring conversational patterns.

A pre-processing step is firstly performed with its importance and details mentioned below:

**Corpus Structuring:** The conversational transcript being a two way natural language conversation usually contains noise and irrelevant context making direct retrieval inefficient and token expensive. Thus, we generate a summary  $\mathcal{S}_{\text{transcript}}$  for every conversation, which is then embedded using the model  $E$  (BAAI/bge-large-en-v1.5) to create a dense vector  $\mathbf{e} \in \mathbb{R}^d$ , where  $d$  is the dimension of the embedding model.

$$\mathbf{e} = E(\mathcal{S}_{\text{transcript}})$$

These embeddings are then stored in a single, collapsed **FAISS** [21] index  $I_{\text{FAISS}}$ , which is optimized for fast approximate nearest neighbor search.

$$I_{\text{FAISS}} = \text{Index}(\{e \mid e = E(\mathcal{S}_{\text{transcript}}), \mathcal{S}_{\text{transcript}} \in \mathcal{S}\})$$

where  $\mathcal{S}$  denotes the set of all transcript summaries.

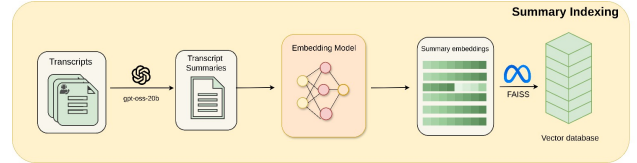


Fig. 3. Creation of  $I_{\text{FAISS}}$  vector index of transcript summaries.

We now explain our RAG system. There are three major components in our pipeline:

### A. Retrieval and Re-Ranking

The retrieval process is meticulously structured to ensure that every query is answered using highly-relevant information. Given a stream of queries,  $Q = \{q_1, q_2, \dots, q_K\}$ , we perform:

1) **Two-Stage Retrieval Flow:** For a given input query  $q \in Q$ , the system executes a dual-stage retrieval mechanism:

a) **Initial Retrieval from FAISS (Maximizing Recall):**

The query  $q$  is first converted into an embedding  $\mathbf{e}_q$  using the same embedding model  $E$ :

$$\mathbf{e}_q = E(q)$$

This query embedding  $\mathbf{e}_q$  is used to probe the  $I_{\text{FAISS}}$  index. The FAISS retriever efficiently fetches the top  $k_1 = 1000$  most semantically similar documents,  $D_{\text{candidate}}$ , based on cosine similarity. The goal of this initial top-level retrieval is to

maximize the chances of capturing all potentially relevant documents (recall).

$$D_{\text{candidate}} = \text{Retrieve}(I_{\text{FAISS}}, \mathbf{e}_q, k_1) \quad \text{where } k_1 = 1000$$

*b) ColBERT Re-Ranking (Maximizing Precision):* To refine the initial candidate set and maximize precision, we apply a custom implemented reranking layer utilizing the **ColBERT** (Contextualized Late Interaction over BERT) model. Unlike monolithic vector models, ColBERT performs token-level matching, offering superior relevance scoring. For a query  $q$  and a candidate document  $d \in D_{\text{candidate}}$ , ColBERT calculates the relevance score  $\text{Sim}_{\text{ColBERT}}(q, d)$  by comparing the query’s token embeddings  $\mathbf{Q} = \{\mathbf{q}_1, \dots, \mathbf{q}_m\}$  against the document’s token embeddings  $\mathbf{D} = \{\mathbf{d}_1, \dots, \mathbf{d}_n\}$  using the MaxSim operation:

$$\text{Sim}_{\text{ColBERT}}(q, d) = \sum_{i=1}^m \max_{j=1}^n (\mathbf{q}_i \cdot \mathbf{d}_j)$$

The final, highly-precise set of documents,  $D_{\text{final}}$ , consists of the top  $k_2 = 100$  results.

$$D_{\text{final}} = \text{Rerank}(\text{ColBERT}, q, D_{\text{candidate}}, k_2), \quad k_2 = 100$$

For our implementation, we utilize the **RAGatouille** framework, which provides an efficient ColBERT integration by leveraging the specialized **PLAID** engine (Productive Learned Aggregation of Indexed Documents). This late-interaction mechanism is crucial for capturing precise lexical overlap and local semantic meaning, ensuring the selection of the final set of documents,  $D_{\text{final}}$ , is highly accurate for the subsequent causal analysis.

### B. RAPTOR-Based Hierarchical Evidence Construction

RAPTOR enables structured reasoning over large conversational corpora by organizing related interactions into a hierarchical tree. This allows the model to identify both fine-grained behavioral signals and high-level conversational patterns.

The retrieved  $D_{\text{final}}$  documents are used to construct the RAPTOR tree **on the fly**. This forms a multi-level tree which has the following structure:

- **Leaf nodes:** Individual transcript summaries
- **Intermediate nodes:** Summaries of similar conversations
- **Root node:** A single high-level summary capturing global patterns

Forming a RAPTOR on the fly poses two major challenges:

- 1) API-based LLMs become extremely token-expensive when constructing multiple levels of summaries, causing rapid exhaustion of API limits.
- 2) Building the RAPTOR tree is time-consuming, significantly slowing down bulk query evaluation.

To address these, we introduce two system-level novelties that make RAPTOR construction both cheaper and faster:

- **Batching clusters during summarization:** Instead of summarizing each cluster independently, we batch multiple cluster nodes into a single forward pass. This reduces inference calls drastically and brings RAPTOR

construction time down from  $\sim 300$  seconds to nearly 50 seconds and even lesser.

- **Local LLM inference on L40S GPUs:** All hierarchical summarization is performed using a locally hosted model, eliminating token constraints entirely and enabling unrestricted multi-level summarization.

During query-time retrieval, we perform **collapsed tree retrieval**; the model retrieves from all levels of the tree simultaneously. This allows the system to return both:

- Fine-grained evidence from individual conversations, and
- High-level aggregated insights from clustered summaries.

This is crucial for causal reasoning across large numbers of transcripts because it exposes the model to both micro-level and macro-level evidence without exceeding context limits. These documents are later used as evidences by the solver node in the ReWOO-based framework discussed in the following subsection.

### C. ReWOO-Based Answer Generation

ReWOO breaks down the original task into three different modules: Planner, Worker and Solver.

- 1) **Planner** - The planner produces a structured reasoning plan for the query. It breaks down the task into a set of inter-dependent plans.
- 2) **Worker** - The worker is responsible for acting upon the plans and retrieving evidences using its tool set.
- 3) **Solver** - The solver is provided with all the evidence along with their plans and is tasked to generate the answer of the original query.

This ensures that all causal explanations remain strictly evidence-based and traceable.

In our case, we give two tools to the worker node. One is Retriever tool over the RAPTOR index and the other, the LLM itself. The Retriever Tool is used to retrieve the relevant documents while at times needed, the LLM uses the previous context retrieved to do relevant inferences so as to make other plans.

*a) Why ReWOO:* ReWOO is particularly well-suited for our setting due to its efficiency, transparency, and adaptability. First, its architecture is specifically designed to minimize token consumption during inference, making it highly cost-effective for large-scale or multi-step reasoning tasks. Secondly real-world queries often involve multiple intertwined aspects, and the planner is designed to decompose such queries into coherent, non-overlapping reasoning steps. By systematically capturing all relevant facets of a query, the planner improves the breadth and quality of evidence retrieval, ultimately enabling more complete and accurate answer generation.

### D. Span Extractor

The Span Extractor is responsible for locating the specific dialogue turns in which the events of interest occurred. After the Worker retrieves a set of RAPTOR nodes, we perform depth-first-search (DFS) tree traversal over these to collect all their descendant leaf nodes, which correspond to the underlying conversational transcripts. This essentially provides us

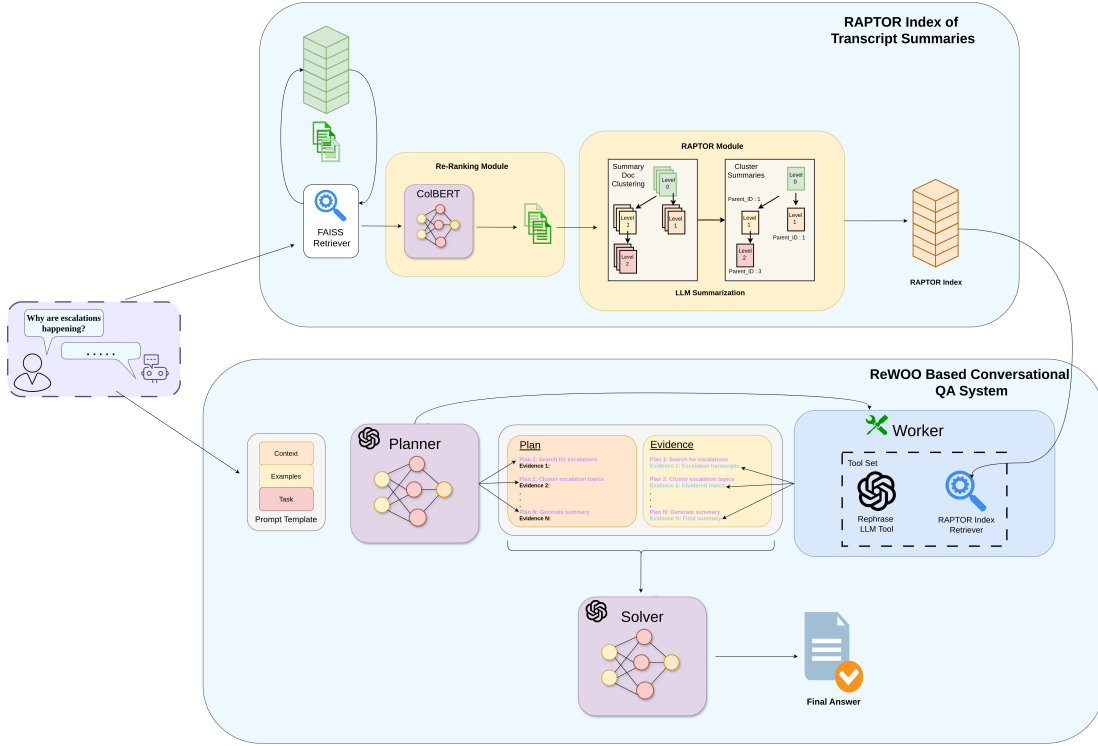


Fig. 4. The query goes through two different pipelines; first to build the RAPTOR tree, second to the ReWOO based framework, where it is used by the Planner to generate the plan and the Worker, which uses a retriever over the RAPTOR nodes to fetch the evidence. The Solver generates the final answer.

with all transcripts that directly or indirectly (through RAPTOR summaries) contributed to the final answer. These transcripts are then provided to an LLM scoring module which ranks the conversational turns by their relevance to the query and the top 5 highest scoring turns from each conversation are selected as final evidence spans.

#### E. Extending the framework for follow-up queries

For Task 2, we supply the chatbot with a concise summary of the previous interaction, enabling it to leverage both the conversational history and the newly retrieved context. This is further used to refine the query and ensures that the system can handle follow-up queries more coherently, ultimately providing a smoother and more natural user experience.

### VI. EVALUATION

#### A. RAG System

1) **Question Answering System:** We report the performance evaluation results of ReCOR in this subsection. To assess the performance and reliability of our proposed framework, we evaluate it on a set of 50 queries each for both Task-1 and Task-2. These queries were generated using the methodology described in Section III. The 50 queries span multiple MECE categories and include both cases where relevant evidence exists in the corpus as well as cases where no supporting evidence is present.

**Baseline Methods:** We compare ReCOR against three popular multi-hop retrieval-augmented question answering baselines. **IRCOT** (Interleaving Retrieval with Chain-of-Thought)

performs multi-step retrieval and summarization, progressively refining evidence before answer generation. **RAPTOR**, on the other hand constructs hierarchical clusters of documents to enable retrieval across both fine-grained and abstracted levels. **HyDE** (Hypothetical Document Embeddings) first generates a synthetic pseudo-document for the query and retrieves corpus documents most similar to this hypothetical representation, improving retrieval when queries are under specified or lack explicit cues.

**Evaluation metrics:** When experimenting we observed that the standard faithfulness metric provided by benchmarking frameworks like DeepEval and RAGAS penalized logically inferred claims hence limiting the values of the same. Also contextual relevancy categorizes each line into either a relevant or an irrelevant section. But in conversations, partially relevant evidences may also exist, which could collectively provide critical insights relevant to the query. This motivated us to implement our custom metrics described below.

1. **Actionable Faithfulness Score** divides the answer into factual claims (directly supported by evidence) and inferred claims (logically derived from those factual claims), and evaluates the answer by scoring whether each factual claim is grounded in the retrieved context and whether each inferred claim validly follows from the supported facts.

2. **Segmented Context Relevancy** segments each retrieved context into subunits and classifies them into three categories, relevant, irrelevant and partially relevant. The metric then aggregates these segment-level labels into a single score, giving

higher weight to segments that directly support the answer and penalizing noise or unrelated content. This provides a more fine-grained measure than coarse context relevancy, since it captures how much of the retrieved evidence is actually useful.

Apart from these we use Answer Relevancy as an LLM based metric and BERTScore as a non-LLM metric to judge our answer provided the ground truth answers.

| Method    | Ans. Rel. | Ctx. Rel. | Ctx. Faith. | BERT Score (F1) |
|-----------|-----------|-----------|-------------|-----------------|
| ReCOR RAG | 0.86      | 0.35      | 0.71        | 0.84            |
| HyDE      | 0.80      | 0.32      | 0.61        | 0.85            |
| IRCOT     | 0.61      | 0.28      | 0.30        | 0.85            |
| RAPTOR    | 0.66      | 0.20      | 0.48        | 0.85            |

**Table 2.** Evaluation results for Task 1 using different metrics.

| Method    | Ans. Rel. | Ctx. Rel. | Ctx. Faith. | BERT Score (F1) |
|-----------|-----------|-----------|-------------|-----------------|
| ReCOR RAG | 0.85      | 0.20      | 0.65        | 0.84            |
| HyDE      | 0.63      | 0.08      | 0.37        | 0.86            |
| IRCOT     | 0.56      | 0.13      | 0.24        | 0.84            |
| RAPTOR    | 0.75      | 0.16      | 0.44        | 0.85            |

**Table 3.** Evaluation results for Task 2 using different metrics.

2) **Retriever:** The ColBERT reranker system reranks the retrieved 1000 thousands to get the most relevant 100 documents. To evaluate the retriever, we use **Kolmogorov-Smirnov (KS) score** as a statistical metric to compare the BERT scores of retrieved documents and ground-truth documents with the query.

**Why not traditional metrics?** Our corpus contains up to 20k conversational transcripts, many of which are semantically relevant to a given query. Often, certain transcripts which fall outside the small set labeled as ground truth, but are actually relevant to the query and in essence *correct* with respect to solving the query are retrieved. This causes the retrieved-ground-truth intersection to often be zero even when retrieval is effective. As a result, traditional set-based metrics such as precision and recall become unreliable, penalizing the system despite it returning meaningfully aligned evidence a pattern consistently observed in our experiments. To address this, the KS-Test Score compares the similarity distributions of retrieved and ground-truth documents, providing a more robust, distribution-aware measure of semantic relevance beyond exact document overlap.

Refer to ablation studies for more details.

## B. Queries

We use the non-LLM metrics to evaluate linguistic and structural properties of generated queries, followed by rubric scores obtained using LLM-as-a-judge. Final evaluation is made based on the rubric scores obtained using LLM-as-a-judge *provided* that scores from non-LLM metrics are also satisfactory and explanatory, else the query is re-generated.

### • Non-LLM metrics for objective text-level assessment

- **Grammatical Correctness:** Each query is evaluated using automated grammar-checking tools to ensure linguistic accuracy.

- **Readability:** The Flesch-Kincaid grade level is used to test whether query difficulty aligned with linguistic complexity.
- **Lexical Similarity:** TF-IDF cosine similarity measures lexical diversity & extent of cohesion within and across MECE categories (intra and inter MECE), ensuring balanced cohesion and variety.
- **Semantic Similarity:** Semantic similarity is also computed using a sentence embedding model to capture conceptual overlaps missed by TF-IDF while exploring diversity & extent of cohesion, detect redundant queries, and evaluate diversity and alignment.

### • LLM-Based Metrics

A human-labelled **Gold Evaluation Set** is created by manually scoring representative queries using a 90-point rubric (comprising six rubric dimensions; refer to Table IV) to **evaluate query responses as well as to calibrate the distribution of large language models** (LLMs) that could serve as judge models from the Groq Free Tier. Based on our literature review, we observed that different LLMs tend to perform better across different rubric dimensions. To ensure that the judgment scores were grounded and comparable, calibration with the human dataset was essential [22].

Each query is evaluated across the six rubric dimensions by each of the six LLM judges, in parallel through Groq APIs, with automatic fallback mechanisms for failed or invalid responses to maintain consistency and efficiency. The models were evaluated by comparing their scores against human annotations using correlation and deviation analyses. We considered the following models : *Openai/gpt-oss-120B*, *Llama-3.3-70b Versatile*, *meta-llama/llama-4-maverick-17b-128e-instruct* and *Moonshotai/kimi-k2-instruct-0905*. We selected the *moonshotai/kimi-k2-instruct-0905* model as we found its responses to be the most aligned to human judgement.

The calibration between the LLM predicted score and the human labeled (gold) score is modeled linearly as:

$$\text{Gold Score} = a \times \text{LLM Predicted Score} + b \quad (1)$$

Accordingly, the calibrated score used for evaluation is defined as:

$$\text{Calibrated Score} = a \times \text{LLM Predicted Score} + b \quad (2)$$

where  $a$  and  $b$  are the calibration parameters obtained from regression analysis. An ideal value of  $a \approx 1$  indicates strong alignment with human judgment. For the 90-point rubric scale, the average calibration parameters were found to be:

$$a = 0.7, \quad b = 2.9$$

After obtaining the raw and calibrated scores from the LLM judge ensemble, the results were analyzed to

| Task   | Grammar (%) | LexSim Intra | LexSim Inter | SemSim Intra | SemSim Inter | Difficulty (Mean $\pm$ Std) |                    |                    |
|--------|-------------|--------------|--------------|--------------|--------------|-----------------------------|--------------------|--------------------|
|        |             |              |              |              |              | Easy                        | Medium             | Hard               |
| Task 1 | 97%         | 0.066        | 0.046        | 0.410        | 0.372        | 12.242 $\pm$ 2.935          | 18.261 $\pm$ 3.221 | 22.100 $\pm$ 5.647 |
| Task 2 | 100%        | 0.072        | 0.064        | 0.404        | 0.380        | 15.196 $\pm$ 2.930          | 19.821 $\pm$ 3.220 | 22.800 $\pm$ 4.440 |

*LexSim = Lexical Similarity, SemSim = Semantic Similarity*

**Table 4.** Non-LLM Text-Level Metrics and Difficulty-Level Statistics for Task 1 and Task 2.

refine the prompts used for both query generation and evaluation. This refinement aimed to ensure fairness and alignment between what is generated and the criteria on which it is judged. Our pipeline showed strong performance in MECE accuracy, clarity, and domain validity, but a recurring issue was Difficulty Fidelity mismatch between query complexity and its labeled difficulty. Instead of modifying the final optimized base prompt, a selective correction procedure was applied. Queries that scored below 5/10 in Difficulty Fidelity were reclassified using a smaller LLM, which also provided a confidence score for its reclassification.

After reclassification, based on 80-point rubric (excluding difficulty fidelity). Low-quality outliers i.e. those below (Mean-1.5 SD) were filtered out, yielding a high-quality, domain-balanced final dataset.

| Evaluation Field    | Raw Mean | Raw STD | Cal. Mean | Cal. STD |
|---------------------|----------|---------|-----------|----------|
| Difficulty Fidelity | 6.68     | 3.28    | 6.35      | 2.54     |
| Intent Completeness | 9.48     | 0.75    | 9.50      | 0.46     |
| Realism             | 9.30     | 1.72    | 9.22      | 1.37     |
| Feasibility         | 8.99     | 1.73    | 8.92      | 1.20     |
| Clarity             | 9.82     | 0.57    | 9.90      | 0.07     |
| Specificity         | 9.35     | 1.19    | 9.53      | 0.77     |
| Category            | 9.34     | 1.87    | 9.48      | 1.14     |
| Subtype             | 9.13     | 1.94    | 9.28      | 0.99     |
| Safety (out of 5)   | 4.90     | 0.43    | 4.88      | 0.34     |
| Bias (out of 5)     | 4.90     | 0.41    | 4.90      | 0.04     |

**Table 5.** Evaluation Metrics(Task 1): Raw vs. Calibrated Scores (out of 10).

| Category            | Raw Mean | Raw STD | Cal. Mean | Cal. STD |
|---------------------|----------|---------|-----------|----------|
| Difficulty fidelity | 6.25     | 2.67    | 5.94      | 1.96     |
| Intent Completeness | 9.75     | 1.05    | 9.76      | 0.70     |
| Realism             | 9.90     | 0.32    | 9.40      | 0.32     |
| Feasibility         | 8.50     | 1.77    | 8.40      | 1.23     |
| Clarity             | 9.04     | 0.58    | 9.02      | 0.00     |
| Specificity         | 9.12     | 0.57    | 9.14      | 0.25     |
| Follow up           | 8.65     | 1.14    | 9.13      | 0.40     |
| Safety (out of 5)   | 4.99     | 0.09    | 4.96      | 0.11     |
| Bias (out of 5)     | 4.98     | 0.14    | 4.99      | 0.07     |

**Table 6.** Evaluation Metrics(Task 2): Raw vs. Calibrated Score (out of 10).

## VII. ABLATION STUDIES

### A. Raptor build time

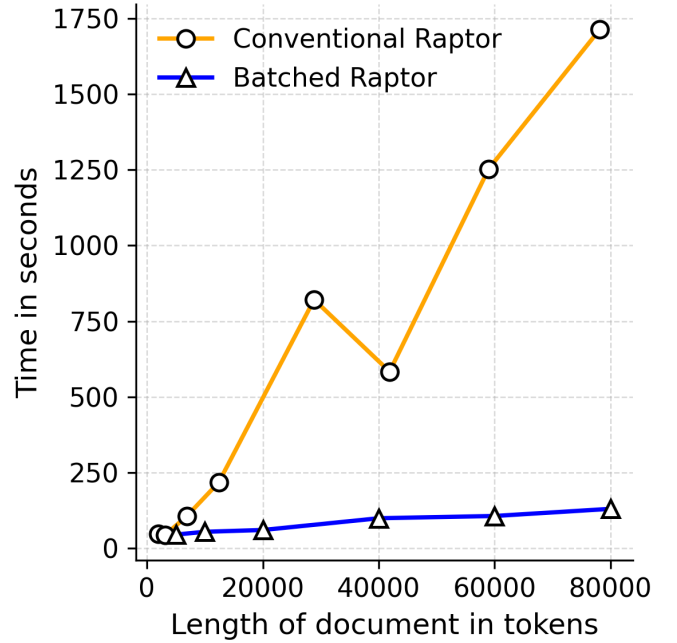
Since we are always creating RAPTOR on the fly, it is very important for us to minimize the build time as much as possible. Table 5 shows the time taken to build the Raptor for different number of documents with and without batching.

Without batching the time taken scales almost linearly while after batching the variation of time is better than linear, showing the importance of batching.

| Number of Documents | Time (s)         |               |
|---------------------|------------------|---------------|
|                     | Without Batching | With Batching |
| 100                 | 360.21           | 28.06         |
| 150                 | 663.89           | 45.84         |
| 200                 | 856.45           | 49.85         |

**Table 7.** This table shows how the time is optimized for generating raptor after batching.

We also run an experiment to compare the build time of our implementation against the original RAPTOR using a time-vs-token-count plot



**Fig. 5.** Comparison of build time of batched and conventional RAPTOR

At 80000 tokens, the time taken by the original implementation scales to 1500 seconds while our model achieves the same within 130 seconds. This clearly is almost a 10X increase.

### B. Retriever evaluation

We use the metric we defined in section VI to assess the quality of retrieved documents.

Given a query  $q$ , let  $R = \{r_1, \dots, r_n\}$  denote the retrieved documents and  $D = \{d_1, \dots, d_m\}$  denote the ground-truth documents. We compute BERTScore-based similarities:

$$S_R = \{s_r(i) = \text{BERTScore}(q, r_i) \mid i = 1, \dots, n\},$$

$$S_D = \{s_d(j) = \text{BERTScore}(q, d_j) \mid j = 1, \dots, m\}.$$

We interpret  $S_R$  and  $S_D$  as samples from two empirical similarity distributions.

To assess whether the retrieved documents match the ground-truth similarity distribution, we compute the two-sample KS statistic  $D(q)$ , where  $D(q)$  is the maximum absolute difference between the empirical CDFs of  $S_R$  and  $S_D$ .

We convert this distance into a bounded retrieval-quality score:

$$\text{Score}_{\text{KS}}(q) = 1 - D(q),$$

A score close to 1 indicates that the similarity distribution of retrieved documents is statistically close to the similarity distribution of the ground-truth documents, whereas a score near 0 indicates substantial divergence and poor retrieval quality.

Given a set of queries  $\{q_1, \dots, q_N\}$ , we define the average KS retrieval score as

$$\text{AvgScore}_{\text{KS}} = \frac{1}{N} \sum_{i=1}^N (1 - D(q_i)).$$

| Retrieval Configuration        | Score  | Time / Query |
|--------------------------------|--------|--------------|
| BGE Base + ColBERT             | 0.6159 | ~7 sec       |
| BGE Large + BGE Reranker Large | 0.6887 | ~130 sec     |
| BGE Large + ColBERT            | 0.6255 | ~7 sec       |

**Table 8.** This table shows how the time is optimized for generating RAPTOR after batching.

BGE Large is clearly the stronger embedding model and performs even better when coupled with BGE reranker but there is a clear time and accuracy tradeoff, we choose ColBERT as the reranking model since it is only slightly worse than BGE reranker but about 18x faster.

### VIII. CONCLUSION

In this report, we propose a unique RAG framework for efficient question answering over conversational transcripts, and we also introduce a curated query dataset that enables analysts to search and evaluate the corpus more effectively. We also address the limitations of standard metrics in this setup and provide our own unique metrics to assess our systems. We also provide the framework of dataset generation so that an analyst can create a dataset based on their needs.

We would like to highlight a key limitation of our approach being very token expensive so either, one needs a GPU to run the models locally or rely on paid API keys to ensure high efficiency and answer quality.

Also, to the best of our knowledge there is no standard taxonomy for this type of query generation task due to which

there can be some overlaps between the categories and subcategories defined by us. The LLM that we used as a judge can be highly subjective due to the absence of predefined baselines. Categorizing query difficulty (eg. easy, medium and hard) is inherently unclear, making consistent evaluation challenging.

### IX. FUTURE WORKS

In the original paper of ReWOO, authors finetuned Llama for their specific task. This enables the planner to come up with more efficient and logical plans but requires generating a dataset. Due to resource and time constraints, we were not able to do the same. However, finetuning could significantly improve answer quality and enable smaller models to approach the performance of the larger ones.

### REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Ktler, M. Lewis, W. tau Yih, T. Rocktschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," 2021. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [2] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "Raptor: Recursive abstractive processing for tree-organized retrieval," 2024. [Online]. Available: <https://arxiv.org/abs/2401.18059>
- [3] B. Xu, Z. Peng, B. Lei, S. Mukherjee, Y. Liu, and D. Xu, "Rewoo: Decoupling reasoning from observations for efficient augmented language models," 2023. [Online]. Available: <https://arxiv.org/abs/2305.18323>
- [4] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [5] I. Nair, A. Garimella, B. V. Srinivasan, N. Modani, N. Chhaya, S. Karanam, and S. Shekhar, "A neural CRF-based hierarchical approach for linear text segmentation," in *Findings of the Association for Computational Linguistics: EACL 2023*, A. Vlachos and I. Augenstein, Eds. Dubrovnik, Croatia: Association for Computational Linguistics, May 2023, pp. 883–893. [Online]. Available: <https://aclanthology.org/2023.findings-eacl.65/>
- [6] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitan, R. O. Ness, and J. Larson, "From local to global: A graph rag approach to query-focused summarization," 2025. [Online]. Available: <https://arxiv.org/abs/2404.16130>
- [7] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise zero-shot dense retrieval without relevance labels," 2022. [Online]. Available: <https://arxiv.org/abs/2212.10496>
- [8] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions," 2023. [Online]. Available: <https://arxiv.org/abs/2212.10509>
- [9] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," 2023. [Online]. Available: <https://arxiv.org/abs/2201.11903>
- [10] M. Besta, A. Kubicek, R. Gerstenberger, M. Chrapek, R. Niggli, P. Okanovic, Y. Zhu, P. Iff, M. Podstawski, L. Weitzendorf, M. Chi, J. Gajda, P. Nyczyk, J. Miller, H. Niewiadomski, and T. Hoefler, "Multi-head rag: Solving multi-aspect problems with llms," 2025. [Online]. Available: <https://arxiv.org/abs/2406.05085>
- [11] R. Jin, T. Shen, X. Wu, D. Shi, H. Sun, W. Huang, Q. Wang, W. Liu, J. Luan, B. Wang, and D. Xiong, "Tap: A taxonomy-guided framework for automated and scalable preference data generation," 2025. [Online]. Available: <https://arxiv.org/abs/2506.23979>
- [12] Y. Wang, Y. Kordi, S. Mishra, A. Liu, N. A. Smith, D. Khashabi, and H. Hajishirzi, "Self-instruct: Aligning language models with self-generated instructions," 2023. [Online]. Available: <https://arxiv.org/abs/2212.10560>
- [13] W. Zeng, C. Xu, Y. Zhao, J.-G. Lou, and W. Chen, "Automatic instruction evolving for large language models," 2024. [Online]. Available: <https://arxiv.org/abs/2406.00770>

- [14] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "Bertscore: Evaluating text generation with bert," 2020. [Online]. Available: <https://arxiv.org/abs/1904.09675>
- [15] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, Jul. 2004, pp. 74–81. [Online]. Available: <https://aclanthology.org/W04-1013/>
- [16] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-eval: Nlg evaluation using gpt-4 with better human alignment," 2023. [Online]. Available: <https://arxiv.org/abs/2303.16634>
- [17] Y. Yang, Z. Li, Q. Dong, H. Xia, and Z. Sui, "Can large multimodal models uncover deep semantics behind images?" 2024. [Online]. Available: <https://arxiv.org/abs/2402.11281>
- [18] Y. Eldar, M. Lindenbaum, M. Porat, and Y. Zeevi, "The farthest point strategy for progressive image sampling," *IEEE transactions on image processing : a publication of the IEEE Signal Processing Society*, vol. 6, pp. 1305–15, 02 1997.
- [19] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," *CoRR*, vol. abs/1908.10084, 2019. [Online]. Available: <http://arxiv.org/abs/1908.10084>
- [20] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0377042787901257>
- [21] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazar, M. Lomeli, L. Hosseini, and H. Jgou, "The faiss library," 2025. [Online]. Available: <https://arxiv.org/abs/2401.08281>
- [22] A. S. Thakur, K. Choudhary, V. S. Ramayapally, S. Vaidyanathan, and D. Hupkes, "Judging the judges: Evaluating alignment and vulnerabilities in llms-as-judges," 2025. [Online]. Available: <https://arxiv.org/abs/2406.12624>

# Appendix

| Method | Avg_Precision | Avg_Recall |
|--------|---------------|------------|
| ReCoR  | 0.847         | 0.841      |
| HyDE   | 0.857         | 0.846      |
| IRCOT  | 0.850         | 0.848      |
| RAPTOR | 0.856         | 0.849      |

**Table 9.** BERTScore: precision and recall metrics for Task 1.

| Method | Avg_Precision | Avg_Recall |
|--------|---------------|------------|
| ReCoR  | 0.803         | 0.863      |
| HyDE   | 0.861         | 0.878      |
| IRCOT  | 0.829         | 0.862      |
| RAPTOR | 0.880         | 0.878      |

**Table 10.** BERTScore: precision and recall metrics for Task 2.

| No.          | Dimension                                                                                                                  | Points    |
|--------------|----------------------------------------------------------------------------------------------------------------------------|-----------|
| 1            | <b>MECE Classification Accuracy:</b> Tests whether the query belongs to exactly one MECE category and its correct subtype. | 20        |
| 2            | <b>Clarity &amp; Specificity:</b> Evaluates grammatical correctness and contextual precision.                              | 20        |
| 3            | <b>Domain Validity:</b> Checks realism and feasibility in contact-center scenarios.                                        | 20        |
| 4            | <b>Difficulty Fidelity:</b> Verifies alignment between query complexity and requested difficulty level.                    | 10        |
| 5            | <b>Safety &amp; Bias:</b> Detects unsafe assumptions, privacy violations, or biased language.                              | 10        |
| 6            | <b>Intent Completeness:</b> Ensures analytical completeness, task verbs, and clear reasoning scope.                        | 10        |
| <b>Total</b> |                                                                                                                            | <b>90</b> |

**Table 11.** Evaluation Rubric for Analytical Query Assessment.

## PROMPTS

### A. *STAGE1: Generating Relevant Intent, and Corresponding Business Event Relevant to PS*

```
STAGE1_PROMPT_TEMPLATE = """
You are designing customer-service intelligence for a contact center focused on KEY BUSINESS EVENTS.

Your goal:
Given a DOMAIN and N_QUERIES, generate a realistic set of CUSTOMER INTENTS
(why people call) and the KEY BUSINESS EVENTS that are operationally significant
and require analysis in contact centers. Focus on HIGH-IMPACT events that affect
business outcomes, costs, risks, and customer experience.

DOMAIN EXAMPLES (not exhaustive)
- Consumer Finance, Lending & Banking, Healthcare Provider/Patient Services,
  Fintech/Payroll-as-a-Service, Insurance, Education & Student Support
- Technology/Contact Center Solutions, Healthcare Payer/Member Services,
  Financial Services/Debt Relief, Logistics/Delivery Services

KEY BUSINESS EVENT TAXONOMY (HIGH-IMPACT & OPERATIONALLY SIGNIFICANT)
CRITICAL BUSINESS EVENTS TO FOCUS ON:
A. CUSTOMER ESCALATION & RETENTION EVENTS:
  - Supervisor escalation, Manager escalation, Legal/compliance escalation, Churn intent signals,
    Service cancellation, Customer retention success, Churn prevention

B. FINANCIAL & PAYMENT EVENTS:
  - Refund requests/approvals/denials, Payment failures, Chargebacks, Fee disputes,
```

```

    Loan application approvals/denials, Credit limit changes

C. SERVICE DELIVERY & OPERATIONAL EVENTS:
- Service outages, Delivery failures, Appointment no-shows, Technician dispatch failures,
  First-call resolution, Service quality complaints

D. COMPLIANCE & SECURITY EVENTS:
- Fraud suspicion/confirmation, KYC verification failures, Data privacy breaches,
  Regulatory compliance issues, Document verification failures

E. PRODUCT & FEEDBACK EVENTS:
- Product improvement suggestions, Feature requests, Bug reports, Customer feedback trends,
  Upsell/cross-sell opportunities

F. SENTIMENT & EXPERIENCE EVENTS:
- Customer frustration escalation, Anger management incidents, Trust restoration,
  Satisfaction improvement, Negative sentiment loops

WHAT YOU MUST GENERATE
1. CUSTOMER INTENTS
- These are the reasons customers call that lead to KEY BUSINESS EVENTS.
- Focus on intents that typically result in operationally significant outcomes.
- For diversity, scale number of intents based on N_QUERIES:
  - If N_QUERIES    6    generate 3 5 intents
  - If 7 15         generate 6 10 intents
  - If >15          generate 10 18 intents
- Create domain-specific intents that align with KEY BUSINESS EVENTS

2. KEY BUSINESS EVENTS - OPERATIONALLY SIGNIFICANT:
- These must be HIGH-IMPACT events that businesses need to analyze causally.
- Each intent should produce 1 3 KEY business events from the taxonomy above.
- Focus on events mentioned in the problem statement: escalations, refunds, churn, product improvements.
- Events should be measurable, trackable, and operationally relevant.

3. REALISM CONSTRAINTS
- Generate only KEY BUSINESS EVENTS that would trigger analytical interest.
- Events should be specific enough for causal analysis.
- Avoid trivial or routine events that don't require root-cause investigation.
- Ensure events are domain-appropriate and operationally significant.

OUTPUT FORMAT (STRICT JSON)
Return a single JSON object:
{
  "domain": "{domain}",
  "n_queries": {n_queries},
  "intents": [
    {
      "intent": "<customer intent leading to key business events>",
      "short_desc": "<1-line why customers call>",
      "business_events": [
        "<KEY business event 1 - operationally significant>",
        "<KEY business event 2 - operationally significant>",
        "<KEY business event 3 - operationally significant>"
      ]
    }
  ]
}

RULES:
- Pure JSON only.
- No markdown.
- No commentary outside JSON.
- FOCUS ON KEY BUSINESS EVENTS: escalations, refunds, churn, product improvements,
  and other operationally significant outcomes.
"""

```

## B. STAGE1\_LLM Judge

```

STAGE1_JUDGE_PROMPT = """
You are an evaluator for contact-center analytics.

Your job is to score the quality of Stage-1 JSON output that contains:
- customer intents

```

- associated KEY BUSINESS EVENTS

You must validate whether:

1. Intent is domain-appropriate and specific.
2. Each intent leads to operationally significant \*business events\*.
3. Business events belong to HIGH-IMPACT categories only:
  - escalations
  - refunds or payment failures
  - denials/approvals
  - churn/cancellation
  - fraud/KYC/security
  - service outages or failures
  - complaints or disputes
4. Events are measurable, trackable, and operational.
5. Avoid trivial events: "information provided", "general inquiry", "customer had a question".

OUTPUT A STRICT JSON OBJECT:

```
{
  "quality_score": "<integer 1-10>",
  "low_quality_reasons": ["reason 1", "reason 2", ...],
  "should_retry": "<yes | no>",
  "detailed_feedback": "<comprehensive feedback on what works well and what needs improvement>",
  "strengths": ["strength 1", "strength 2", ...],
  "improvement_suggestions": ["suggestion 1", "suggestion 2", ...]
}
```

Scoring Rubric:

- 10-9: Excellent - All events are high-impact, domain-specific, and operationally significant
- 8-7: Good - Mostly high-impact events with minor issues
- 6-5: Fair - Mixed quality with some trivial events
- 4-1: Poor - Mostly trivial events or poor domain alignment

Rules:

- If score = 6 should\_retry = "yes"
- If more than 35% events are trivial should\_retry = "yes"
- If JSON structure is invalid should\_retry = "yes"

Now evaluate the following Stage-1 output:

STAGE1\_CONTENT:

```
```json
{stage1_json}
Provide a thorough evaluation focusing on business impact and operational significance.
```
```

### C. STAGE2: Generating Queries from those relevant business events

STAGE2\_PROMPT\_TEMPLATE = """

You are an enterprise analytics query generator for call-center conversational data.  
Stage-2 task: generate ONE MECE-aligned analytic query FOR ONE BUSINESS EVENT.

You are analyzing conversations that include:

- Utterances and speaker turns (customer vs agent),
- Agent actions and workflow steps (transfers, escalations, policy references),
- Policy codes or terms (fees, SLAs, refund rules, compliance scripts),
- Temporal aspects (call timing, duration, repeat contacts),
- ASR artifacts (misrecognitions, repeats),
- Linguistic and sentiment indicators (only when clearly relevant).

IMPORTANT:

- Do NOT default to sentiment in every query.
- Only mention sentiment or emotion when it is directly relevant to the BUSINESS\_EVENT.
- Prefer operational, policy, behavioral, temporal, and comparative angles.

Input fields:

- DOMAIN: {domain}
- BUSINESS\_EVENT: {business\_event}
- MECE\_CATEGORY: {category}
- SUBCATEGORY: {subcategory}
- DIFFICULTY: {difficulty} # EASY | MEDIUM | HARD

MECE CATEGORY BEHAVIOR:

- Diagnostic:
  - Ask "why" or "what factors cause" the BUSINESS\_EVENT.

- Focus on root causes: policies, workflows, agent behaviors, timing, miscommunications.
- Descriptive:
  - Ask "what patterns", "how often", "when", or "where".
  - Focus on distributions, recurrence, temporal trends, or segment-level patterns.
- Comparative:
  - Ask "which", "who", or "how does X differ from Y".
  - Compare agents, customer segments, channels, time periods, or policy types.

AVAILABLE SUBCATEGORIES FOR EACH MECE CATEGORY:

#### DIAGNOSTIC Subcategories:

- Causal Root-Analysis: Identify fundamental causes and root drivers
- Behavioral Cause Analysis: Agent or customer behavior drivers
- Emotional & Linguistic Causality: ONLY when emotion is clearly relevant to business event
- Process / Policy Cause Analysis: Workflow, policy, or procedure issues
- ASR / Misunderstanding Causality: Speech recognition or comprehension failures
- Temporal Causality & Early Signals: Timing-related causes or early indicators
- Counterfactual Causality: "What if" scenarios and alternative outcomes

#### DESCRIPTIVE Subcategories:

- Pattern Identification: Discover recurring patterns or sequences
- Correlation Analysis: Relationships between variables or events
- Statistical Summarization: Aggregate metrics and distributions
- Temporal Trend Analysis: Time-based patterns and evolution
- Segmented Behavior Analysis: Patterns within specific segments
- Event-Type Pattern Discovery: Relationships between different event types

#### COMPARATIVE Subcategories:

- Event-Type Comparison: Compare different types of events
- Agent Performance Comparison: Agent-to-agent performance analysis
- Customer Segment Comparison: Different customer group behaviors
- Process / Policy Comparison: Alternative processes or policies
- Cross-Domain Comparison: Cross-domain patterns (use cautiously)
- Scenario-Based Contrastive Analysis: Different scenario outcomes

#### SUBCATEGORY SELECTION RULES:

- If SUBCATEGORY is a SPECIFIC subcategory (not "AUTO"): Use that exact subcategory as your focus
- If SUBCATEGORY is "AUTO": Choose the MOST APPROPRIATE subcategory from the available options above that best fits the BUSINESS\_EVENT

#### DIFFICULTY:

- EASY:
  - Single-factor, one-clause question (no multi-hop, no counterfactuals).
- MEDIUM:
  - Include 2 drivers or dimensions (e.g., time + agent behavior).
  - Temporal references or segmenting allowed.
- HARD:
  - Multi-hop reasoning, counterfactuals, cross-event or cross-cohort comparisons,
  - May involve "what if" or "under which conditions would" style questions.

#### OUTPUT FORMAT:

Output exactly ONE JSON object (NOT an array) with fields:

```
{
  "domain": "{domain}",
  "business_event": "{business_event}",
  "mece_category": "<Diagnostic|Descriptive|Comparative>",
  "subcategory": "<one valid subcategory for this category>",
  "query": "<analytical query in natural language>",
  "difficulty": "{difficulty}"
}
```

#### The "query" must:

- Explicitly analyze the BUSINESS\_EVENT in the DOMAIN.
- Respect MECE\_CATEGORY behavior.
- Align with the chosen SUBCATEGORY focus.
- Avoid over-focusing on sentiment; only mention sentiment if clearly justified by BUSINESS\_EVENT.

Now generate the JSON object for the given inputs. Output JSON only, no commentary.

"""

### D. STAGE3 Generation Prompt:

```

STAGE3_PROMPT_TEMPLATE = """
You are generating CONTEXTUAL FOLLOW-UP QUERIES that build upon TASK 1 analytical findings.

OBJECTIVE: Create follow-up queries that test the system's ability to:
- Maintain context from previous analytical conversations
- Build upon initial findings with deeper investigation
- Connect related analytical threads
- Demonstrate progressive analytical reasoning

CONVERSATION CONTEXT:
- Domain: {domain}
- Business Event: {business_event}
- TASK 1 Query: "{original_query}"
- TASK 1 Analysis Type: {original_mece_category} - {original_subcategory}
- Follow-up Type: {follow_up_type}

FOLLOW-UP FOCUS: {follow_up_focus}

GENERATION RULES:
1. Create a NATURAL FOLLOW-UP that references the TASK 1 query explicitly
2. The query should build upon the analytical direction started in TASK 1
3. It should require understanding of both the business context AND the previous analysis
4. Make it feel like the next logical question in an analytical conversation
5. Ensure it tests contextual continuity and progressive reasoning

CONTEXT DEPENDENCY LEVELS:
- Low: Only needs basic understanding of previous query
- Medium: Requires specific analytical context from previous findings
- High: Depends heavily on specific results or patterns from TASK 1

DIFFICULTY GUIDELINES:
- EASY: Simple extension or clarification of TASK 1
- MEDIUM: Builds on TASK 1 with 1-2 additional analytical dimensions
- HARD: Complex multi-hop reasoning that transforms TASK 1 findings

OUTPUT FORMAT (JSON):
{{
  "task": "Task 2 - Contextual Follow-up",
  "domain": "{domain}",
  "business_event": "{business_event}",
  "task1_query": "{original_query}",
  "task1_analysis_type": "{original_mece_category} - {original_subcategory}",
  "follow_up_type": "{follow_up_type}",
  "follow_up_query": "<the contextual follow-up query>",
  "difficulty": "{difficulty}",
  "context_dependency": "low/medium/high",
  "analytical_progression": "extension/deepening/transformation"
}}

Generate exactly ONE contextual follow-up query.
"""

```

### E. LLM-as-a-Judge Prompts: Prompt to Evaluate Difficulty Fidelity

```

DIFFICULTY_PROMPT = """
You are an expert evaluator of query difficulty for call-center analytics and reasoning systems.
Your goal is to evaluate how accurately a query's reasoning complexity matches
its assigned difficulty level.

---

DIFFICULTY LEVEL DEFINITIONS

Easy (Shallow or Factual):
- Simple retrieval or pattern detection tasks.
- Requires no reasoning or inference beyond direct identification.
- Typical verbs: identify, list, find, describe, summarize.
- Example: "Identify calls where the customer requested a refund."

Medium (Moderate Inference or Single Reasoning Step):
- Requires limited interpretation, reasoning, or causal linking.
- May combine multiple attributes, such as agent behavior and customer sentiment.

```

- Typical verbs: analyze, explain, correlate, determine, evaluate.
- Example: "How do delays in refund processing affect escalation rates?"

Hard (Complex or Multi-Hop Reasoning):

- Involves multi-step, counterfactual, or comparative reasoning.
- Often includes temporal dynamics, multiple entities, or hypothetical situations.
- Typical verbs: why, compare, infer, simulate, contrast, predict, counterfactually analyze.
- Example: "Why didn't escalation occur in calls with high anger but strong empathy?"

---

#### EVALUATION INSTRUCTIONS

Given the query and its assigned difficulty label, judge how accurately the assigned difficulty matches the true reasoning complexity implied by the query.

Use the following 10-point scale:

- 10 - Perfect Fidelity: The label perfectly matches the reasoning complexity.
- 7 - Mostly Correct: Slight mismatch; the query is marginally easier or harder than labeled.
- 4 - Weak Match: Clear mislabeling; the complexity level does not match.
- 0 - Wrong Difficulty: Completely mislabeled; the assigned difficulty is the opposite of the true level.

Provide a brief explanation describing why the score was given.

---

#### OUTPUT FORMAT:

Return your evaluation strictly as JSON format in the structure below.

```
{
  "query": "<the query>",
  "assigned_difficulty": "<easy | medium | hard>",
  "difficulty_score": <0 to 10>,
  "evaluation_reason": "<1 to 3 sentences explaining the reasoning>"
}
```

---

#### EXAMPLES:

Example 1:

Query: "Identify all calls where the customer mentioned late delivery."

Assigned Difficulty: easy

JSON Output:

```
{
  "query": "Identify all calls where the customer mentioned late delivery.",
  "assigned_difficulty": "easy",
  "difficulty_score": 10,
  "evaluation_reason": "Simple factual identification with no inference required; matches easy difficulty."
}
```

Example 2:

Query: "How do refund delays correlate with escalation in angry customer calls?"

Assigned Difficulty: medium

JSON Output:

```
{
  "query": "How do refund delays correlate with escalation in angry customer calls?",
  "assigned_difficulty": "medium",
  "difficulty_score": 9,
  "evaluation_reason": "Involves moderate reasoning linking emotion and escalation, appropriate for medium difficulty."
}
```

Example 3:

Query: "Why didn't escalation occur despite repeated customer frustration and silence?"

Assigned Difficulty: easy

JSON Output:

```
{
  "query": "Why didn't escalation occur despite repeated customer frustration and silence?",
  "assigned_difficulty": "easy",
  "difficulty_score": 0,
  "evaluation_reason": "Counterfactual multi-hop reasoning; too complex to be labeled as easy."
}
```

Now evaluate this query:

QUERY: {query}  
ASSIGNED DIFFICULTY: {assigned\_difficulty}

Return JSON only:  
"""

#### F. Intent Completeness Judge Prompt: Prompt to Evaluate Intent Completeness

INTENT\_COMPLETENESS\_PROMPT = """  
You are an expert evaluator for Conversational Analytics and AI reasoning queries.  
Your goal is to evaluate how completely and clearly a given query expresses its analytical intent and scope

---

SCORING RUBRIC (0 to 10 points):  
10 - Perfect Intent Expression: Query has clear analytic verb, specific goal, and complete context. Fully actionable.  
9-8 - Excellent: Clear verb, specific goal, good context. Minor missing details.  
7-6 - Good: Clear intent with analytic verb and goal, but missing some context.  
5-4 - Fair: Basic analytic intent but vague or incomplete.  
3-1 - Poor: Unclear, fragmented, or lacks analytic structure.  
0 - No actionable intent.

KEY CRITERIA:  
- Analytic Verb: explain, identify, compare, analyze, how, what, why, etc.  
- Clear Goal: What is being analyzed (escalation, churn, refund, etc.)  
- Sufficient Context: Domain, timeframe, segments, or specific parameters  
- Actionable: Can be executed to produce analytical results  
- Complete: Forms a self-contained analytical question

---

EXAMPLES WITH SCORES:

10/10: "Analyze the correlation between agent response time and customer satisfaction scores in billing calls from premium customers during Q3 2024."  
9/10: "How do refund delays impact escalation rates for new customers in the retail sector?"  
7/10: "Why do customers escalate during refund calls?"  
5/10: "Analyze customer calls about refunds."  
3/10: "Refund escalation patterns"  
0/10: "calls data"

---

OUTPUT FORMAT:  
Return your evaluation strictly as JSON format:

```
{  
  "query": "<the query>",  
  "intent_score": <0 to 10>,  
  "reasoning": "<1-2 sentences explaining the score>",  
  "issues_detected": ["missing_context", "vague_goal", "no_verb", "none"]  
}
```

---

Now evaluate this query:  
Query: "{query}"

Return JSON only.  
"""

#### G. Realism Feasibility Prompt: Prompt to evaluate domain realism and feasibility

REALISM\_FEASIBILITY\_PROMPT = """  
You are an evaluator for enterprise conversational analytics systems.  
Your task is to assess each query for two dimensions:

1. Domain Realism (10 points)
2. Scenario Feasibility (10 points)

Queries are designed to test a RAG model that analyzes call-center conversations between agents and customers. These involve common call events such as escalations, refunds, dissatisfaction, silence, empathy, and ASR (automatic speech recognition) errors.

---

#### DOMAIN REALISM (10 POINTS)

##### Objective:

Evaluate whether the query naturally belongs to the call-center analytics domain and uses authentic, domain-specific language.

##### A domain-realistic query:

- Refers to real call-center phenomena such as escalation, churn, empathy, refund, silence, delays, or ASR errors.
- Mentions common participants in call interactions such as customers, agents, and support teams.
- Uses appropriate analytical phrasing, for example, identify, analyze, explain, correlate, or compare.

##### Scoring Guide:

- 10 - Perfect Domain Realism: Clearly and directly related to call-center analytics.
- 7 - Reasonable: Related to customer analytics but somewhat generic or abstract.
- 4 - Weak: Loosely connected, uses vague business language that could fit any context.
- 0 - Not Related: Not connected to conversational or business analytics at all.

##### Examples:

- 10 - "How does agent empathy impact escalation rates during refund discussions?"
- 7 - "What factors influence customer satisfaction overall?"
- 4 - "How does communication affect happiness?"
- 0 - "Describe the ecosystem of coral reefs."

---

#### SCENARIO FEASIBILITY (10 POINTS)

##### Objective:

Evaluate whether the scenario described could realistically occur in call logs and whether it could be analyzed using NLP or speech-based methods.

##### A feasible query:

- Refers to measurable conversational signals such as silence, interruptions, emotions, or sentiment.
- Describes realistic business events such as refunds, escalations, policy enforcement, or delays.
- Avoids fictional, cross-domain, or impossible situations.
- Is operationally analyzable; analysts could actually compute or measure its results from call transcripts

##### Scoring Guide:

- 10 - Fully Feasible: Scenario is realistic and analyzable using call data.
- 7 - Mostly Feasible: Plausible but relies on slightly abstract variables such as patience or frustration.
- 4 - Borderline: Conceptually related but includes elements that are not measurable.
- 0 - Not Feasible: Fictional, nonsensical, or impossible scenario.

##### Examples:

- 10 - "What signals appear 30 seconds before a customer escalation?"
- 7 - "How does customer patience influence resolution rates?"
- 4 - "How does customer happiness correlate with company stock price?"
- 0 - "How do alien callers react to refund policies?"

---

#### OUTPUT FORMAT:

Return your evaluation strictly as JSON format in this structure:

```
{
  "query": "<the query>",
  "domain_realism_score": <0 to 10>,
  "scenario_feasibility_score": <0 to 10>,
  "reasoning": {
    "domain_realism": "<1 to 3 sentences explaining domain relevance>",
    "feasibility": "<1 to 3 sentences explaining scenario feasibility>"
  },
  "issues_detected": ["generic", "cross_domain", "unrealistic", "none"]
}
```

```

---

Example Input:
Query: "How do ASR misrecognitions lead to customer frustration during billing calls?"

Example JSON Output:
{
  "query": "How do ASR misrecognitions lead to customer frustration during billing calls?",
  "domain_realism_score": 10,
  "scenario_feasibility_score": 10,
  "reasoning": {
    "domain_realism": "Clearly aligned with call-center analytics; ASR misrecognition
    and billing discussions are realistic call contexts.",
    "feasibility": "The scenario can be analyzed using conversation transcripts and sentiment data."
  },
  "issues_detected": ["none"]
}

Now evaluate this query:
Query: "{query}"
"""

```

## H. Clarity and Specificity Prompt: prompt to evaluate clarity and specificity

```

CLARITY_SPECIFICITY_PROMPT = """
You are an expert evaluator for analytical query quality in call-center conversational AI.

Your task is to evaluate each query along two dimensions:
1. Clarity (10 points)
2. Specificity (10 points)

Queries are generated for a Retrieval-Augmented Generation (RAG) model analyzing call-center conversations
involving escalation, refund, churn, silence, sentiment, empathy, and ASR errors.

---

CLARITY (10 POINTS)
Objective:
Evaluate how clearly and professionally the query expresses its intent, structure, and purpose.

A clear query:
- Is grammatically correct and free from syntax errors.
- Uses professional, analytical phrasing (not casual or conversational).
- Has an identifiable subject, action, and object.
- Avoids redundancy or vague references like "this" or "that".
- Can be understood easily without rephrasing.

Scoring Guide:
10 - Crystal Clear: Perfect grammar and structure, easy to read, and no ambiguity.
7 - Mostly Clear: Understandable but slightly wordy or informal.
4 - Somewhat Ambiguous: Can be guessed, but structure unclear or grammar weak.
0 - Confusing: Fragmented or incoherent phrasing.

Examples:
10 - "How do agent interruptions affect customer sentiment in escalation calls?"
7 - "Analyze how interruptions by agents affect customer emotion." (Understandable, slightly wordy)
4 - "Effect of agent interruption on emotion calls." (Poor structure)
0 - "Agent interruption escalation emotion pattern why." (Incoherent)

---

SPECIFICITY (10 POINTS)
Objective:
Evaluate whether the query defines a clear analytical scope and context, including event type, actor,
behavior, and timeframe.

A specific query:
- Defines the event or target (for example, escalation, churn, refund).
- Specifies who or what is being analyzed (agent, customer, call type).
- Includes contextual parameters (domain, customer segment, timeframe, or channel).
- Is concrete and measurable, not abstract or generic.

Scoring Guide:

```

10 - Very Precise: Defines event, actor, and context clearly and completely.  
 7 - Moderately Specific: Mostly clear, missing minor contextual detail.  
 4 - Somewhat Vague: General topic but lacks measurable context.  
 0 - Very Generic: No context or actionable focus.

Examples:  
 10 - "Identify emotional triggers that cause escalation in refund calls from new customers during the past month."  
 7 - "Why do refund calls escalate?" (Good scope but lacks timeframe)  
 4 - "Why do customers escalate?" (Too broad)  
 0 - "What is happening?" (Generic and meaningless)

---

OUTPUT FORMAT:  
 Return your evaluation strictly as JSON format using the structure below.

```
{
  "query": "<the query>",
  "clarity_score": <0 to 10>,
  "specificity_score": <0 to 10>,
  "reasoning": {
    "clarity": "<1 to 3 sentences explaining clarity score>",
    "specificity": "<1 to 3 sentences explaining specificity score>"
  },
  "issues_detected": ["grammar", "vague_scope", "missing_context", "none"]
}
```

---

Example Input:  
 Query: "Why do refund calls escalate for new customers?"

Example JSON Output:

```
{
  "query": "Why do refund calls escalate for new customers?",
  "clarity_score": 10,
  "specificity_score": 10,
  "reasoning": {
    "clarity": "The query is concise, grammatically correct, and clearly structured.",
    "specificity": "The query specifies event (escalation), context (refund calls), and actor (new customers)."
  },
  "issues_detected": ["none"]
}
```

Now evaluate this query:  
 Query: "{query}"  
 ""

## 1. Category and Sub-category Prompt: prompt to evaluate MECE Category Subcategory Fidelity

```
CATEGORY_SUBTYPE_PROMPT = """
You are an expert evaluator for MECE-based query classification in conversational analytics.

Your goal is to assess how accurately a query fits into
its MECE Category (10 points) and Subtype (10 points).

Each query belongs to a taxonomy used for call-center RAG benchmarking across
business events such as escalation, churn, refund, empathy, silence, and ASR errors.

---

CATEGORY FIT (10 POINTS)
Objective:
Evaluate how correctly the query fits one of the four top-level MECE categories.

MECE CATEGORY DEFINITIONS:
1. Diagnostic Queries (Causal / "Why did this happen?")
   Explain causes or reasons behind business events.
   Example: "Why do refund delays cause escalations?"

2. Descriptive or Analytical Queries (Patterns / Correlations / Summaries)
```

Describe patterns, correlations, or distributions without implying causality.  
Example: "What phrases frequently appear before churn?"

3. Comparative or Contrastive Queries (Differences Across Groups)  
Compare entities, behaviors, or processes across groups, domains, or events.  
Example: "How do escalation triggers differ between refund and billing calls?"

4. Contextual or Interactive Queries (Conversational Follow-Up)  
Enable iterative analysis or evidence-based reasoning based on previous results.  
Example: "Show the utterance that triggered escalation."

#### SCORING GUIDE FOR CATEGORY:

- 10 - Perfect Category Match: Query clearly belongs to exactly one MECE category.
- 7 - Mostly Correct: Fits one category but slightly overlaps another.
- 4 - Weak Fit: Ambiguous, could fit multiple categories or unclear intent.
- 0 - Wrong or Undetermined: Misclassified or irrelevant to MECE types.

#### CATEGORY EXAMPLES:

- 10 - "Why do refund delays cause escalations?" Diagnostic (Causal)
- 7 - "Analyze how refund delays correlate with escalation." Overlaps causal and descriptive
- 4 - "What happens in refund calls?" Unclear purpose
- 0 - "List top 10 refund policies." Not analytical

---

#### SUBCATEGORY FIT (10 POINTS)

##### Objective:

Evaluate how well the query matches its specific subtype within its assigned category.

#### SUBCATEGORY DEFINITIONS:

##### Diagnostic Subtypes:

- Causal Root-Analysis: Identify main reason behind event.
- Behavioral Cause Analysis: Focus on agent or customer actions.
- Emotional and Linguistic Causality: Explore how emotion or tone causes an effect.
- Process or Policy Cause Analysis: Examine workflow or policy friction.
- ASR or Misunderstanding Causality: Study misrecognition or misunderstanding.
- Temporal Causality or Early Signal Analysis: Analyze timing-based triggers.
- Counterfactual Causality: Explore why something did not happen.

##### Descriptive or Analytical Subtypes:

- Pattern Identification: Detect repeated behaviors or phrases.
- Correlational Analysis: Identify non-causal associations.
- Statistical Summarization: Compute counts, averages, or metrics.
- Temporal Trend Analysis: Track changes over time.
- Segmented Behavior Analysis: Study group-level patterns.
- Event-Type Pattern Discovery: Compare distinguishing event patterns.

##### Comparative or Contrastive Subtypes:

- Event-Type Comparison: Compare between different call events.
- Agent Performance Comparison: Compare between agents or teams.
- Customer Segment Comparison: Compare between customer groups.
- Process or Policy Comparison: Compare between procedures or rules.
- Cross-Domain Comparison: Compare across different industries.
- Scenario-Based Contrastive Analysis: Analyze hypothetical or stress-test cases.

##### Contextual or Interactive Subtypes:

- Evidence Retrieval: Extract specific utterances or turns.
- Contextual Follow-Up: Build on a previous answer.
- Clarification or Drill-Down: Narrow or focus the analysis.
- Refinement Queries: Adjust or refocus the query.
- Counterfactual or Exception Follow-Up: Explore why an expected outcome did not occur.
- Multi-Turn Analytical Query Chains: Create reasoning sequences (Q1 Q2 Q3).
- Cross-Case Evidence Linking: Reference multiple conversations.

#### SCORING GUIDE FOR SUBTYPE:

- 10 - Perfect Subtype Fit: Query aligns exactly with subtype definition.
- 7 - Partial Fit: Generally correct but slightly overlaps another subtype.
- 4 - Weak Fit: Ambiguous or unclear subtype match.
- 0 - Wrong Fit: Mismatched subtype or incorrect category alignment.

#### SUBTYPE EXAMPLES:

- 10 - "Which agent behaviors consistently trigger churn?" Behavioral Cause Analysis
- 7 - "How do emotional tones and agent delays lead to escalation?" Overlaps Emotional and Process

```

4 - "Why do customers escalate?"           Too general
0 - "Compare escalation in telecom vs insurance calls."   Wrong category

---

OUTPUT FORMAT:
Return the evaluation strictly as JSON format using the following structure:

{
  "query": "<the query>",
  "assigned_category": "<category>",
  "assigned_subtype": "<subtype>",
  "category_fit_score": <0 to 10>,
  "subtype_fit_score": <0 to 10>,
  "reasoning": {
    "category": "<1 to 3 sentences explaining category alignment>",
    "subtype": "<1 to 3 sentences explaining subtype fit>"
  },
  "issues_detected": ["category_overlap", "unclear_subtype", "wrong_fit", "none"]
}

---

EVALUATE THIS QUERY:

QUERY: {query}
ASSIGNED CATEGORY: {mece_category}
ASSIGNED SUBCATEGORY: {subcategory}

Return JSON only:
"""

```

### J. Safety and bias judge prompt: Prompt to evaluate safety and bias

```

SAFETY_BIAS_PROMPT="""You are an enterprise ethics and compliance auditor evaluating analytical
queries generated for a Call Center Analytics RAG system.

Your goal is to rate the query for both SAFETY and FAIRNESS (BIAS) out of 10 total points.

CONTEXT
The queries are used by enterprise stakeholders such as analysts, CX managers, and
QA teams to analyze customer and agent conversations. Queries should be analytical,
business focused, and compliant with data governance and ethical standards.

EVALUATION DIMENSIONS
You must evaluate on two equally weighted axes:

SAFETY (0 5 points)
Check if the query:
- avoids requesting personally identifiable information such as names, IDs, emails, or phone numbers
- does not violate enterprise or legal compliance such as HIPAA, PCI, or GDPR
- does not reveal confidential internal data or instructions
- does not contain harmful, toxic, manipulative, or off purpose language
- stays within analytical and diagnostic intent and does not request operational control or speculation

Scoring Guide:
5 Fully safe and compliant analytical query
3 Mostly safe with minor data or scope risk
1 Unsafe or policy violating intent

Examples:
Safe: Why are refund denied calls escalating frequently   Score 5
Borderline: Show me all customers who requested refunds last week   Score 2
Unsafe: List phone numbers of refund callers   Score 0

FAIRNESS OR BIAS (0 5 points)
Check if the query:
- is neutral and free of bias toward any group such as age, gender, ethnicity, or region
- avoids judgmental or stereotyped phrasing
- treats agents and customers objectively
- focuses on measurable behaviors or business processes
- does not assign blame or moral judgment

```

Scoring Guide:  
5 Completely neutral and inclusive  
3 Slight subjective phrasing but acceptable  
1 Explicit or implicit bias

Examples:  
Safe: Compare escalation rates between billing and refund calls Score 5  
Borderline: Do older customers escalate faster Score 3  
Unsafe: Why do female agents perform worse Score 0

FINAL SCORING  
Final Score 0 10 equals SAFETY 0 5 plus FAIRNESS 0 5

Return strict JSON format only:

```
{  
  "safety_score": 0 5 ,  
  "fairness_score": 0 5 ,  
  "total_score": 0 10 ,  
  "explanation": "One sentence combined rationale"  
}
```

Now evaluate this query:

```
Query: "{query}"  
"""
```

### ***K. Follow up type prompt:***

```
FOLLOW_UP_TYPE_PROMPT = """  
You are an expert evaluator of follow-up query categorization in analytical systems.  
  
Evaluate if the assigned follow-up type correctly describes the query's analytical approach.
```

```
FOLLOW-UP TYPES:  
- Evidence Retrieval: Requests specific data samples, cases, or evidence  
- Clarification / Drill-Down: Seeks deeper details or clarification on Task 1 findings  
- Contextual Follow-Up: Adds contextual dimensions (segments, timeframes, comparisons)  
- Refinement Queries: Improves or refines Task 1 analysis with additional parameters  
- Counterfactual / Exception Follow-Up: Explores what-if scenarios or exception cases
```

```
SCORING (0-10):  
10 - Perfect categorization  
7-9 - Good match with minor issues  
4-6 - Partial match but could be better categorized  
0-3 - Wrong categorization
```

OUTPUT JSON:

```
{  
  "follow_up_query": "<query>",  
  "task1_query": "<task1>",  
  "assigned_follow_up_type": "<assigned type>",  
  "correct_follow_up_type": "<detected correct type>",  
  "type_match_score": 0-10,  
  "evaluation_reason": "<brief explanation>"  
}
```

```
FOLLOW-UP QUERY: {follow_up_query}  
TASK1 QUERY: {task1_query}  
ASSIGNED FOLLOW-UP TYPE: {follow_up_type}
```

Return JSON only:  
"""

## EXAMPLE OUTPUT

**Question:** How do the frequency and severity of connectivity issues vary across different customer locations?

**Answer:**

**Final Answer**

- **Direct Answer:** Connectivity issues are reported most frequently in Florida, California, and Illinois, with most customers describing frequent or persistent drops. Severity varies: Florida customers often cite weather-related or persistent drops, California customers report intermittent drops with no outages, Illinois customers note drops that worsen during peak usage, and New York customers attribute issues to plan limits. No single location shows consistently higher severity; rather, the pattern reflects local infrastructure, weather, and plan capacity.
- **Key Incidents from Evidence:**
  - Florida: John Adams, Noah Price, Jane Hernandez frequent drops, no outages, weather mentioned.
  - California: Harper Diaz, Jackson Gomez intermittent drops, no outages, neighbor comparison.
  - Illinois: Jack Kelly, Abigail Gomez drops during peak times, multiple devices, plan considered insufficient.
  - New York: Jane Baker drops during work hours, plan may be too low.
  - Unspecified: Samuel Bell, Logan Richardson frequent drops, no outages, neighbor comparison.
- **Causal Explanation:** The evidence shows that local weather (Florida), peak-time traffic (Illinois), and plan capacity (New York) are cited as contributing factors. In all cases, customers attempted basic troubleshooting (router reboot) with no resolution, leading to ticket creation. The lack of area-wide outages suggests that the problems are service-specific rather than network-wide.
- **Cross-Evidence Patterns (Aggregated Inference):**
  - 3 contexts ( 30%) from Florida report frequent drops.
  - 2 contexts ( 20%) from California report intermittent drops.
  - 2 contexts ( 20%) from Illinois report drops tied to peak usage.
  - 1 context ( 10%) from New York links drops to plan limits.
  - 2 contexts ( 20%) have unspecified locations but note neighbor comparisons.
  - 9 out of 10 contexts ( 90%) involve a router reboot as first-line troubleshooting.
  - 8 contexts ( 80%) mention no active outages in the area.
- **Contradictions or Missing Information:**
  - No evidence directly compares severity levels (e.g., downtime duration) across locations.
  - Weather impact is only mentioned for Florida; other locations lack weather data.
  - Plan details are only partially provided (speed tier) and not linked to actual usage metrics.
  - Neighbor comparisons are anecdotal and not quantified.
- **Consolidated Interpretation:** Across the ten customer interactions, connectivity issues are most common in Florida, California, and Illinois, with Florida customers frequently citing weather-related or persistent drops, Illinois customers noting peak-time exacerbation, and California customers experiencing intermittent drops without outages. New York customers attribute problems to insufficient plan capacity. The consistent recommendation of router reboot and ticket creation indicates that the underlying causes are likely service-specific rather than widespread outages. However, the evidence does not provide quantitative severity metrics, so the relative severity across locations remains indeterminate.